

GitHub Repository: https://github.com/yosammy01/CSC8830_COMPUTER_VISION/tree/main/MODULE56

Web Application: https://yosammy01.github.io/CSC8830_COMPUTER_VISION/#/module56

Consider (capture) any 2 videos of your choice. Consider at least 30 seconds of each video where there is motion involved. For each of the 30sec samples, the following was considered:

- I. Optical Flow
 - a. Compute optical flow and visualize the same as a video.
 - i. The optical_flow.py script uses Lucas-Kanade method to estimate optical flow.
 - ii. This assumes: (1) the motion of pixels is constant within a local pixel neighborhood and (2) pixel intensity of the moving object is constant.
 - iii. Original Videos:

[ball bouncing](#)

[dogs running](#)
 - iv. Videos With Optical Flow Vector Fields Visualized:

[ball bouncing lk vector field](#)

[dogs running lk vector field](#)
 - b. Explain what information can be inferred from the optical flow information? Show evidence.
 - i. Speed and Direction: The vectors show the direction and magnitude of the speed of the pixels. The dog in the example below is clearly identified by the vectors, as moving in the direction of the ball with a fast speed, as indicated by the magnitude of the vectors.
 - ii. Same-Object Identification: Vectors of adjacent pixels pointing in the same direction, with the same magnitude, identify the same object. The vector group located within the boundaries of the body of the dog show the same behavior, indicating they represent the movement of the same object.



Figure 1: Vectors Indicate Direction and Speed

- c. Derive the motion tracking equations from fundamentals and set up the problem of tracking for two image frames.

Motion Tracking Equations

1. Assumptions: (1) It is assumed that the brightness or intensity remains the same throughout the frames. (2) It is assumed the motion between frames is very small.

2. From assumption (1), we can say the change in location of a pixel location, over time, is:

$$* I(x_0, y_0, t_0) = I(x_0 + \Delta x, y_0 + \Delta y, t_0 + \Delta t)$$

3. From assumption (2), we can assume the Taylor Expansion of $I(x_0, y_0, t_0)$ can be written without considering the higher order terms, since they are approximately 0.

$$* I(x_0 + \Delta x, y_0 + \Delta y, t_0 + \Delta t) \approx I(x_0, y_0, t_0) + \left(\frac{dI}{dx} \Delta x + \frac{dI}{dy} \Delta y + \frac{dI}{dt} \Delta t \right)$$

4. Combining the two equations gives:

$$I(x_0, y_0, t_0) = I(x_0 + \Delta x, y_0 + \Delta y, t_0 + \Delta t) + \left(\frac{dI}{dx} \Delta x + \frac{dI}{dy} \Delta y + \frac{dI}{dt} \Delta t \right)$$
$$\frac{dI}{dx} \Delta x + \frac{dI}{dy} \Delta y + \frac{dI}{dt} \Delta t = 0$$

5. Divide both sides by Δt :

$$\frac{dI}{dx} \cdot \frac{\Delta x}{\Delta t} + \frac{dI}{dy} \cdot \frac{\Delta y}{\Delta t} + \frac{dI}{dt} \cdot \frac{\Delta t}{\Delta t} = 0$$

6. Velocity Vectors Defined as:

$$u = \frac{\Delta x}{\Delta t} \quad \text{and} \quad v = \frac{\Delta y}{\Delta t}$$

7. Spatial Gradients Defined as:

$$I_x = \frac{dI}{dx} \quad \text{and} \quad I_y = \frac{dI}{dy} \quad \text{and}$$

$$I_t = \frac{dI}{dt}$$

8. the new equation is:

$$I_x u + I_y v + I_t = 0$$

} ^{unknowns}
two ~~variables~~
+
one equation

9. Lucas-Kanade Spatial Constraint:
Assuming small window of pixels
all move with the same velocity,

$$I_{x_1} u + I_{y_1} v = -I_{t_1}$$

$\vdots$

$$I_{x_{N^2}} u + I_{y_{N^2}} v = -I_{t_{N^2}}$$

10. An $Av=b$ Matrix equation
can be ~~solved~~ using least squares.

$$A = \begin{bmatrix} I_{x_1} & I_{y_1} \\ \vdots & \vdots \\ I_{x_{N^2}} & I_{y_{N^2}} \end{bmatrix}, \quad \checkmark \quad \text{~~u~~} = \begin{bmatrix} u \\ v \end{bmatrix}$$

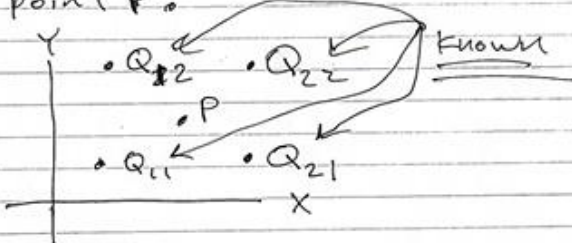
$$b = \begin{bmatrix} I_{t_1} \\ \vdots \\ I_{t_{N^2}} \end{bmatrix}$$

$$v = (A^T A)^{-1} A^T b$$

- d. Derive/describe bilinear interpolation.
- Bilinear Interpolation is used in a coarse-to-fine estimation algorithm in computer vision. This is when the motion tracking equations are not valid due to large movement of pixels. In this case, the resolution is lowered until the motion tracking equations become valid. Then, in a step-by-step manner, the optical flow results of the lower resolution image are interpolated to the next higher resolution level and so forth.
 - It can estimate an unknown 2D point by referencing 4 points close to it that are known.

Bilinear Interpolation

1. Obtain 4 points that are both known and are corners around the unknown point P.



$$f(P) \approx \frac{y_2 - y}{y_2 - y_1} f(R_1) + \frac{y - y_1}{y_2 - y_1} f(R_2)$$

$$f(R_1) \approx \frac{x_2 - x}{x_2 - x_1} f(Q_{11}) + \frac{x - x_1}{x_2 - x_1} f(Q_{21})$$

$$f(R_2) \approx \frac{x_2 - x}{x_2 - x_1} f(Q_{12}) + \frac{x - x_1}{x_2 - x_1} f(Q_{22})$$

$$Q_{11} = (x_1, y_1)$$

$$Q_{12} = (x_1, y_2)$$

$$Q_{21} = (x_2, y_1) \quad Q_{22} = (x_2, y_2)$$

- e. Pick two consecutive frames from each of the videos and validate the theoretical result of tracking with actual pixel locations.
 - i. The `validate_tracking.py` script compares the chosen pixel in a chosen frame with the estimated location of the same pixel in the next frame.
 - ii. The Lucas-Kanade Algorithm sufficiently stayed with an 8-pixel radius of the actual tracked pixel.



Figure 2: Ball Tracking Comparison Result – OpenCV (Green Circle) vs Lucas-Kanade Algorithm (Blue Dot)

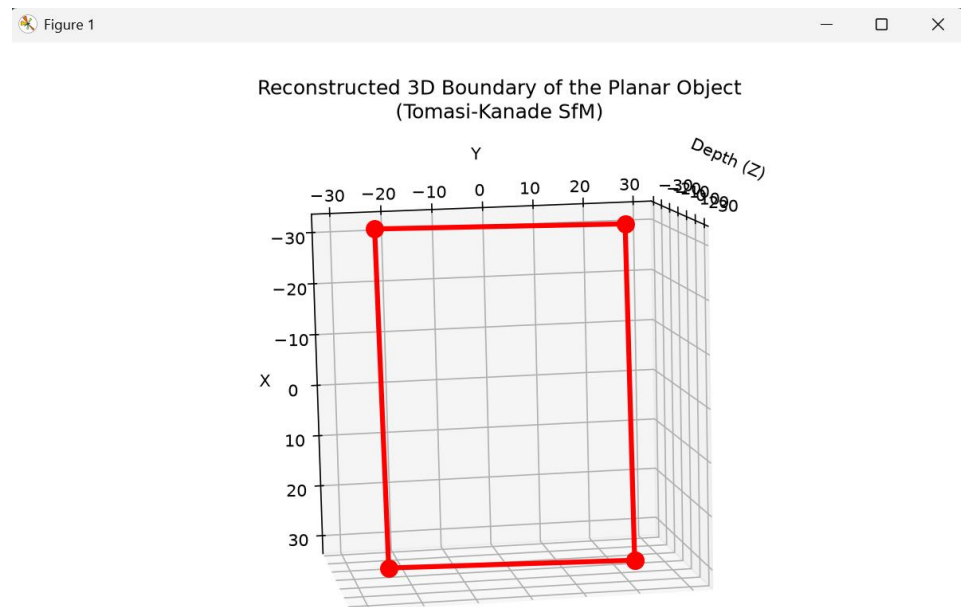


Figure 3: Dog Tracking Comparison Result – OpenCV (Green Circle) vs Lucas-Kanade Algorithm (Blue Dot)

- II. Structure from Motion
 - a. Construct an example of structure from motion using data from four different viewpoints of an object of your interest. Try to recreate the points on the object so that its boundary can be estimated (consider a flat/2D planar object for simplicity).
 - i. The `sfm_planar.py` script uses Matplotlib and Tomasi-Kanade Factorization method to reconstruct the 3D planar boundary of an object.
 - ii. The object used was a book stack.
 - iii. Camera positions were taken with book flat on the table with front facing up.
 - 1. Directly above so only a 2D shape was seen.
 - 2. Above and from the left.
 - 3. Above and from the right.
 - 4. Above and from the bottom.



- iv. The result matched the book 2D front cover boundary.



b. Camera parameters.

Camera	
Camera maker	samsung
Camera model	SM-G991U
F-stop	f/1.8
Exposure time	1/24 sec.
ISO speed	ISO-640
Exposure bias	0 step
Focal length	5 mm
Max aperture	1.69
Metering mode	Center Weighted Average
Subject distance	
Flash mode	No flash
Flash energy	
35mm focal length	26

c. Mathematical workouts and calculations.

i. Feature Tracking and Measurement Matrix

1. I used 4 images of the object from 4 different angles and selected the four corners of the object in each image to form the Measurement Matrix:

$$W = \begin{bmatrix} u_{1,1} & u_{1,2} & u_{1,3} & u_{1,4} \\ v_{1,1} & v_{1,2} & v_{1,3} & v_{1,4} \\ u_{2,1} & u_{2,2} & u_{2,3} & u_{2,4} \\ v_{2,1} & v_{2,2} & v_{2,3} & v_{2,4} \\ u_{3,1} & u_{3,2} & u_{3,3} & u_{3,4} \\ v_{3,1} & v_{3,2} & v_{3,3} & v_{3,4} \\ u_{4,1} & u_{4,2} & u_{4,3} & u_{4,4} \\ v_{4,1} & v_{4,2} & v_{4,3} & v_{4,4} \end{bmatrix}$$

- ii. Data Centering
 1. The average of the selected points in each frame was subtracted from all the points in the frame to shift the origin to the center of the object.
- iii. Singular Value Decomposition
 1. The centered Measurement Matrix was then converted with SVD and then that form was converted into the Motion Matrix and Structure Matrix.
 - a. $W_centered = U * \Sigma * V^T$
- iv. Rank-3 Truncation
 1. Due to the constraint that $W_centered$ is of Rank 3, only the top 3 singular values and their corresponding vectors remain.
 - a. $W_centered = U_3 * \Sigma_3 * V_3^T$
- v. Recovering 3D Structure
 1. Motion and Structure Matrices are obtained by factoring $W_centered$.
 - a. $M = U_3 * (\Sigma_3 \wedge 0.5)$
 - b. $S = (\Sigma_3 \wedge 0.5) * V_3^T$
 2. The Structure Matrix forms the Reconstructed 3D Boundary when plotted on 3D graph.